SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1 Total Marks: 100

Annexure A

Assignment

Code of Conduct: I **D Varshini / 192320007** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Course Code & Name	CSA11 – Object Oriented Analysis and Design
Assignment Title	Object-Oriented Analysis and UML-Based Design of an AI-Enabled Disaster Resilience and Emergency Response Platform (AIDERP)
Course Outcome(s) – CO	CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented

	Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices. CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 3 – Good Health and Well-being SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities SDG 13 – Climate Action

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an AI-Enabled Disaster Resilience and Emergency Response Platform (AIDERP). The platform is intended to support communities, emergency coordinators, hospitals, police/fire services, volunteers, utility teams, and local administrators during floods, cyclones, fires, earthquakes, heatwaves, and other emergencies.

The system shall collect and integrate information from citizen reports, IoT sensors, weather feeds, emergency service updates, geospatial data, and infrastructure status. An intelligent risk-analysis module shall assess the severity and likely impact of an incident and recommend suitable response actions. The platform shall provide role-based interfaces for citizens, emergency operators, field responders, medical teams, volunteers, and administrators.

Citizens shall be able to report incidents, share their location, request emergency assistance, receive verified alerts, identify nearby shelters, and obtain evacuation guidance. Emergency operators shall validate incidents, prioritize cases, coordinate

response teams, monitor resource availability, and issue alerts. Field responders shall receive assignments, update incident status, share observations, and request additional resources. Hospitals and shelters shall update capacity and availability.

The system may use IoT-enabled flood, fire, air-quality, temperature, and structural sensors. A geospatial intelligence module shall support incident mapping, safe-route identification, shelter selection, and resource allocation. AI-based prioritization may consider incident severity, population exposure, accessibility, available resources, and real-time sensor information.

Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop appropriate UML models including Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams. Students shall evaluate object responsibilities and apply suitable design principles and patterns to achieve modularity, maintainability, reusability, scalability, resilience, and secure information exchange.

Constraints

The system shall clearly define the system boundary, actors, objects, classes, attributes, operations, relationships, assumptions, and external interfaces. UML diagrams shall be consistent with one another, and the operations identified in the Class Diagram shall support the interactions represented in the Sequence and Activity Diagrams.

The design shall demonstrate appropriate use of encapsulation, abstraction, inheritance, polymorphism, cohesion, coupling, responsibility assignment, and interface-based design. Suitable design patterns shall be selected and justified based on the identified design problems.

The system shall distinguish between verified and unverified incident information and should address availability, privacy, authentication, authorization, fault tolerance, and graceful degradation during network or infrastructure failures. AI recommendations shall be treated as decision support rather than unquestionable commands.

The final model shall document assumptions and constraints and ensure that all UML models are complete, consistent, unambiguous, traceable, feasible, scalable, and maintainable.

Tools Can Be Used

- StarUML
- Visual Paradigm

- Draw.io
- PlantUML
- Papyrus Tools
- ArgoUML
- Enterprise Architect
- Lucidchart

Suggested Key Scenarios for UML Modelling

- Citizen reports a flood/fire emergency and requests assistance.
- AI risk engine prioritizes incidents and recommends response resources.
- Emergency operator verifies an incident and dispatches a response team.
- Responder accepts an assignment and updates the incident status from En Route to Resolved.
- Citizen receives an evacuation alert and obtains a recommended safe route to a shelter.
- Hospital or shelter updates available capacity during a large-scale emergency.
- IoT sensor detects abnormal conditions and automatically creates a preliminary incident.

ANSWER – AIDERP OBJECT-ORIENTED ANALYSIS AND UML-BASED DESIGN

AI-Enabled Disaster Resilience and Emergency Response Platform

1. Problem Analysis and Requirement Summary

AIDERP is a multi-stakeholder emergency-response platform intended to support disaster preparedness, incident reporting, verification, prioritization, resource coordination, evacuation guidance and recovery monitoring for floods, cyclones, fires, earthquakes, heatwaves and similar emergencies. The platform integrates citizen reports, IoT sensor readings, weather feeds, geospatial information, emergency-service updates and infrastructure status. Its central design challenge is to convert heterogeneous and potentially unreliable real-time inputs into verified, prioritized and actionable information without allowing automated recommendations to replace human responsibility.

The system boundary includes incident intake, validation, risk assessment, geospatial analysis, alerting, dispatch, responder coordination, facility-capacity updates, resource monitoring, role-based access and audit logging. External systems include weather and map services, emergency-agency interfaces, IoT/edge networks and communication channels. The design explicitly distinguishes verified from unverified information and supports degraded operation when network or infrastructure services are unavailable.

Primary Stakeholder Goals

- Citizens: quickly report emergencies, share location, request help, receive verified alerts and identify safe shelters/routes.
- Emergency operators: validate reports, prioritize incidents, allocate resources, dispatch teams and maintain a common operational picture.
- Field responders: receive assignments, navigate safely, update status, report observations and request additional resources.
- Hospitals and shelters: publish capacity and availability so dispatch and evacuation decisions remain current.

- Administrators: manage users, roles, policies, audit records, integrations and system configuration.
- AI/geospatial services: provide decision-support risk scores, route recommendations and resource suggestions with confidence/context rather than unquestionable commands.

2. Functional and Non-Functional Requirements

ID	Functional Requirement	Description
FR1	Incident reporting	Accept citizen reports with type, description, location, time and optional evidence.
FR2	Assistance request	Allow a citizen to request emergency help and track acknowledgement/status.
FR3	Sensor/feed ingestion	Receive IoT, weather, geospatial and infrastructure updates and validate source/timestamp.
FR4	Incident verification	Allow an authorized operator to verify, reject, merge or mark an incident as unverified.
FR5	AI risk assessment	Calculate severity/risk using incident details, exposure, accessibility, resources and real-time feeds.
FR6	Prioritization and dispatch	Rank incidents and dispatch suitable responders/resources after operator approval.
FR7	Responder workflow	Support assignment acceptance, En Route/On Scene/Resolved updates, observations and resource requests.
FR8	Alerts and evacuation	Publish verified alerts and provide safe-route/shelter recommendations.
FR9	Facility/resource updates	Maintain hospital, shelter and resource capacity/availability.
FR10	Identity and audit	Provide authentication, role-based authorization and tamper-evident audit logging.

ID	Quality Attribute	Requirement
NFR1	Availability & resilience	Core incident/dispatch functions should remain available 24/7; failover and graceful degradation are required.
NFR2	Performance	Time-critical operations should respond rapidly; routing and incident retrieval should remain usable during disaster surges.

ID	Quality Attribute	Requirement
NFR3	Scalability	Scale horizontally across many simultaneous incidents, users, sensors and notification events.
NFR4	Security	Encrypt data in transit/at rest; apply MFA for privileged roles, least privilege, secure APIs and audit trails.
NFR5	Privacy	Collect only necessary personal/location data, enforce purpose limitation and retention rules, and restrict sensitive health/location access.
NFR6	Integrity	Clearly label verified/unverified information; validate source, timestamp and consistency before operational use.
NFR7	Maintainability	Use modular layers, cohesive services, interface-based dependencies and explicit contracts.
NFR8	Usability & accessibility	Role-specific interfaces must remain understandable under stress and support accessible presentation on mobile/desktop.
NFR9	Explainable decision support	AI outputs should show risk level, factors/confidence and require human validation for consequential actions.
NFR10	Interoperability	Use stable APIs/events for weather, GIS, IoT and emergency-service integration.

3. Actor Identification

Actor	Type	Responsibilities / Interaction
Citizen	Primary	Reports incidents, requests assistance, shares location, receives alerts, searches shelters/routes.
Emergency Operator	Primary	Verifies incidents, sets priority, coordinates resources, dispatches teams and issues alerts.
Field Responder	Primary	Accepts assignments, updates incident status, reports observations and requests resources.
Medical Team / Hospital	Primary	Receives medical coordination requests and updates bed/service capacity.
Shelter Coordinator	Primary	Updates shelter capacity, accessibility and availability.
Volunteer / Utility Team	Supporting	Receives approved assignments and updates task/resource status.
Local Administrator	Administrative	Manages users, roles, policies, configuration, integrations and audits.
IoT Sensor / Edge Gateway	External	Supplies flood, fire, air-quality, temperature or structural readings.

Actor	Type	Responsibilities / Interaction
Weather / GIS Provider	External	Supplies forecast, hazard layers, road/geospatial and routing information.
Emergency Service System	External	Exchanges incident, dispatch or status information with police/fire/ambulance systems.

4. Use Case Descriptions

Use Case	Primary Actor	Precondition	Main Flow	Postcondition	Alternate / Exception
UC1 – Report Incident	Citizen	Authenticated or guest reporting permitted by policy	Create an incident with location, category, description and evidence	Input is validated; incident is stored as Reported/Unverified and risk analysis is triggered	Invalid location/data, duplicate report, network failure
UC2 – Verify Incident	Emergency Operator	Operator authenticated and authorized	Review evidence, related reports, sensor/feed context and AI assessment	Incident becomes Verified, Rejected or remains Under Review	Insufficient evidence, conflict between sources
UC3 – Prioritize & Dispatch	Emergency Operator	Verified incident exists	Review risk recommendation; choose team/resources; dispatch	Assignment created and responder notified	No resource available -> escalate/recommend alternate
UC4 – Respond to Assignment	Field Responder	Assignment received	Accept; update En Route/On Scene; submit observations; resolve	Incident/assignment status synchronized and audited	Responder unavailable, route unsafe, resource shortage
UC5 – Receive	Citizen	Verified alert or	Get current location;	Safe route and shelter details	No safe route -> show nearest

Use Case	Primary Actor	Precondition	Main Flow	Postcondition	Alternate / Exception
Evacuation Guidance		user requests route	select safe shelter; calculate route using hazard/road data	displayed	safe assembly point/instructions
UC6 – Update Capacity	Hospital/Shelter	Authorized facility user	Update beds/spaces, services and timestamp	Capacity becomes available to operators/risk engine	Stale update -> warning and verification
UC7 – Create Preliminary Incident from Sensor	IoT Sensor / System	Trusted sensor reading arrives	Validate reading; detect abnormal pattern; correlate nearby data	Preliminary incident created and queued for operator verification	Bad signature, stale reading, isolated anomaly -> discard/monitor

5. Use Case Diagram

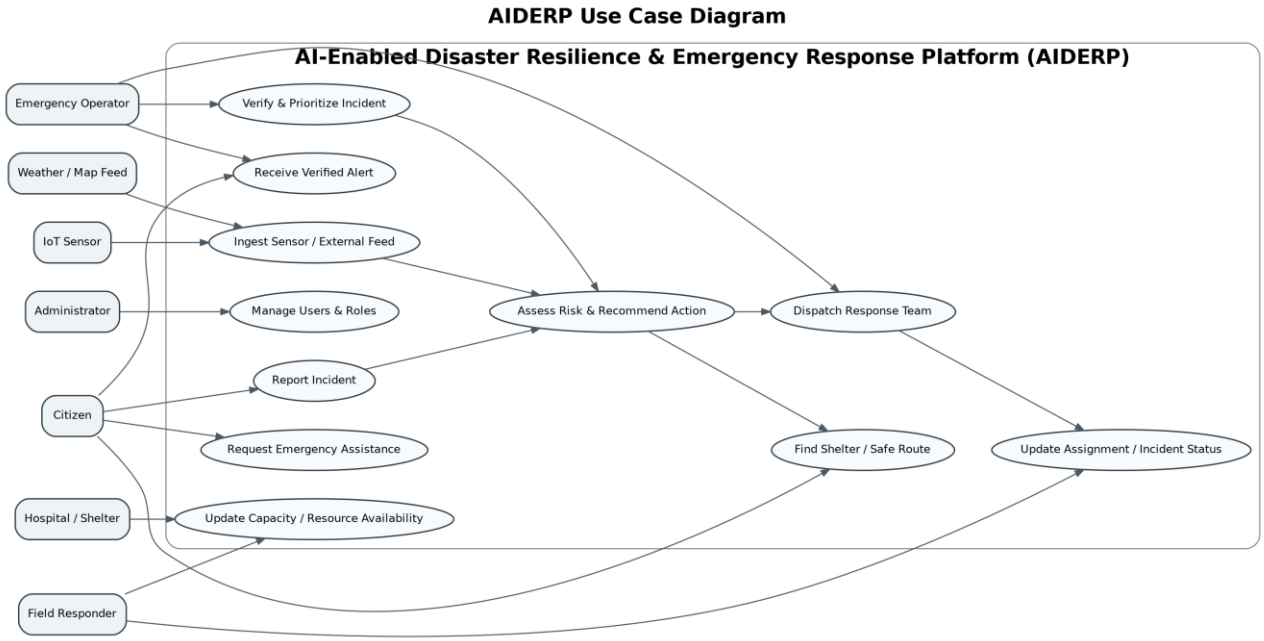

Figure 1. AIDERP Use Case Diagram

The diagram places citizen-facing, operator, responder, facility, administrative and external-feed interactions inside one system boundary. Risk assessment is modeled as a supporting use case invoked by incident intake and operator verification; dispatch depends on verified prioritization rather than directly on raw AI output.

6. Object and Class Identification

Class	Key Attributes	Key Operations	Responsibility
User	userId, name, contact, role	authenticate(), updateProfile()	Abstract parent for role-specific users
Citizen	location	reportIncident(), requestAssistance(), viewAlert()	Specialized User
EmergencyOperator	controlCenterId	verifyIncident(), prioritizeIncident(), dispatchTeam()	Specialized User
Responder	unitType, availability	acceptAssignment(), updateStatus(), requestResource()	Specialized User
Incident	type, severity,	addEvidence(), setStatus(),	Central domain

Class	Key Attributes	Key Operations	Responsibility
	status, location, verified	markVerified()	aggregate
RiskAssessment	riskScore, exposure, confidence	calculateRisk(), recommendActions()	Decision-support result linked to Incident
Assignment	priority, status	assign(), accept(), complete()	Connects Incident, Responder and Resources
Resource	type, quantity, status	reserve(), release(), updateAvailability()	Shared response resource
Facility	capacity, available	updateCapacity(), isAvailable()	Abstract parent for Hospital/Shelter
Alert	message, level, verified	publish(), revoke()	Only verified operational alerts are publishable
SensorReading	type, value, timestamp	validate(), createPreliminaryIncident()	External observation object
SafeRoute	distanceKm, riskLevel	calculate(), recalculate()	Geospatial route to safe facility

7. Class Diagram

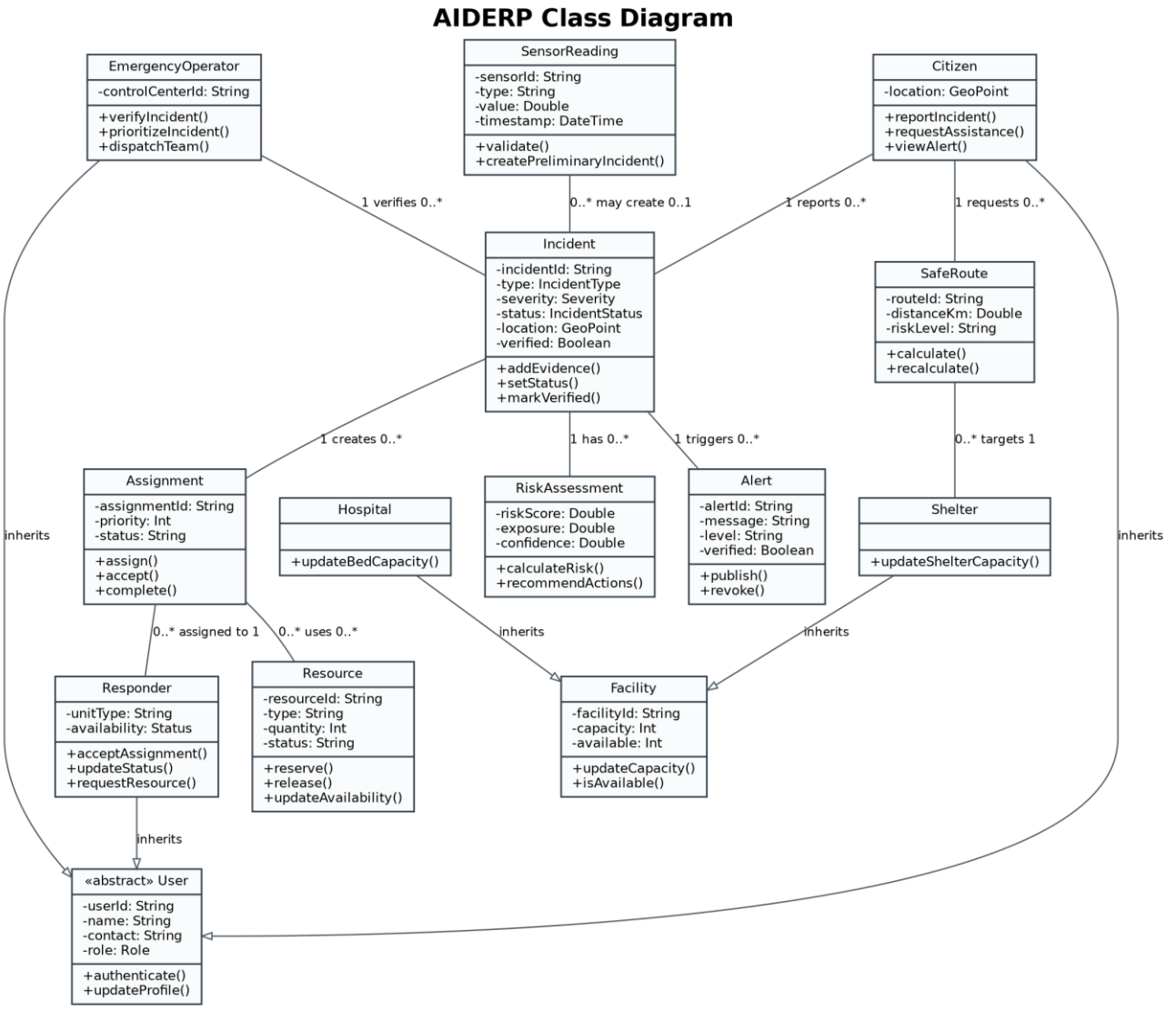

Figure 2. AIDERP Class Diagram with inheritance, associations and multiplicities

The class model uses inheritance only where substitutability is meaningful: Citizen, EmergencyOperator and Responder specialize User; Hospital and Shelter specialize Facility. Incident remains the principal domain object and is associated with assessments, assignments, alerts and evidence. Operations in this diagram are intentionally reused in the sequence/activity models to maintain cross-diagram consistency.

8. Sequence / Interaction Diagrams for Key Emergency Scenarios

8.1 Citizen report, verification and dispatch

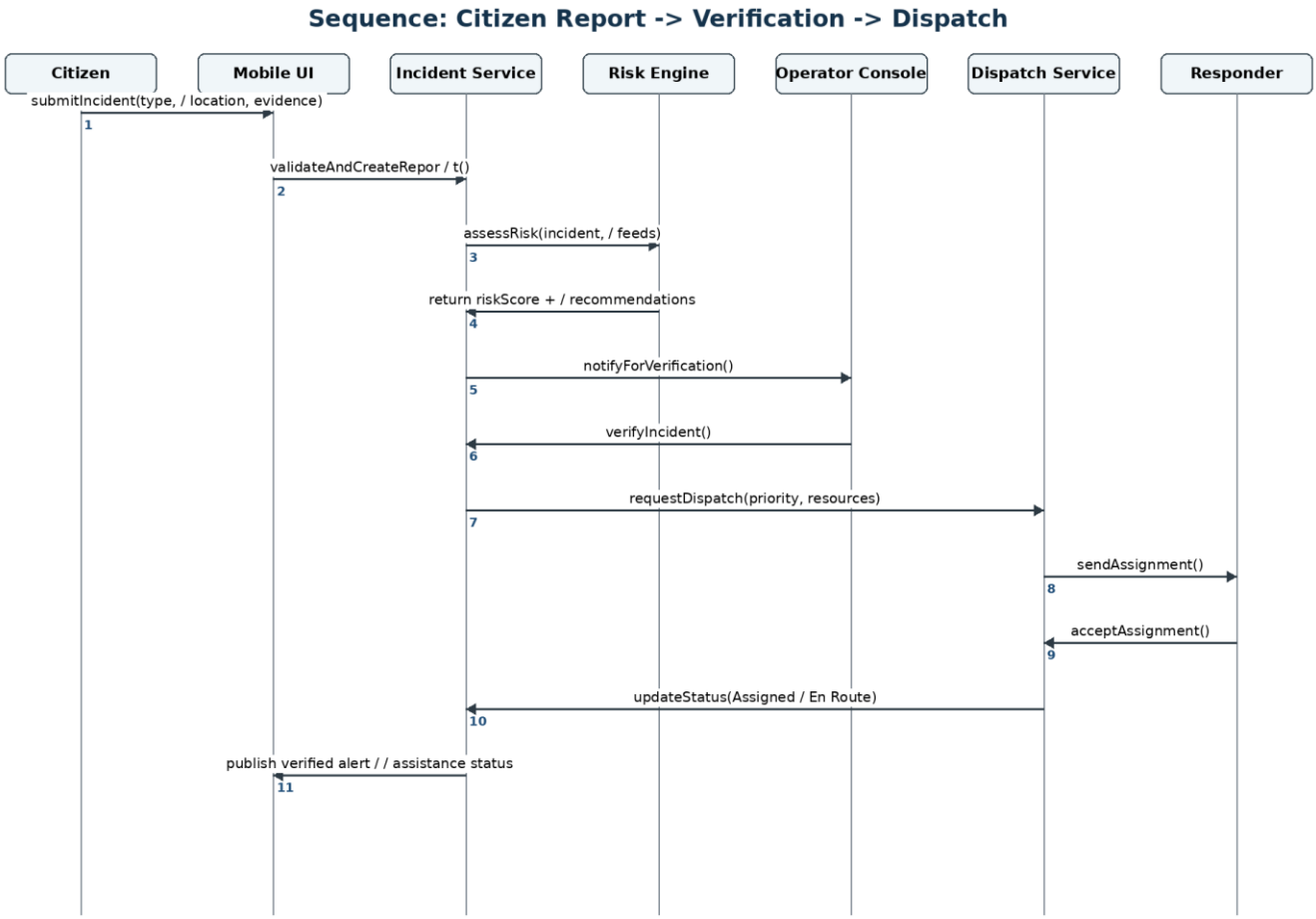

Figure 3. Sequence – Citizen report to verified dispatch

The interaction ensures that a citizen report is validated and assessed before an operator verifies it. The operator, not the AI component, authorizes consequential dispatch. The Dispatch Service then tracks acknowledgement and status updates from the responder.

8.2 IoT abnormal condition creates a preliminary incident

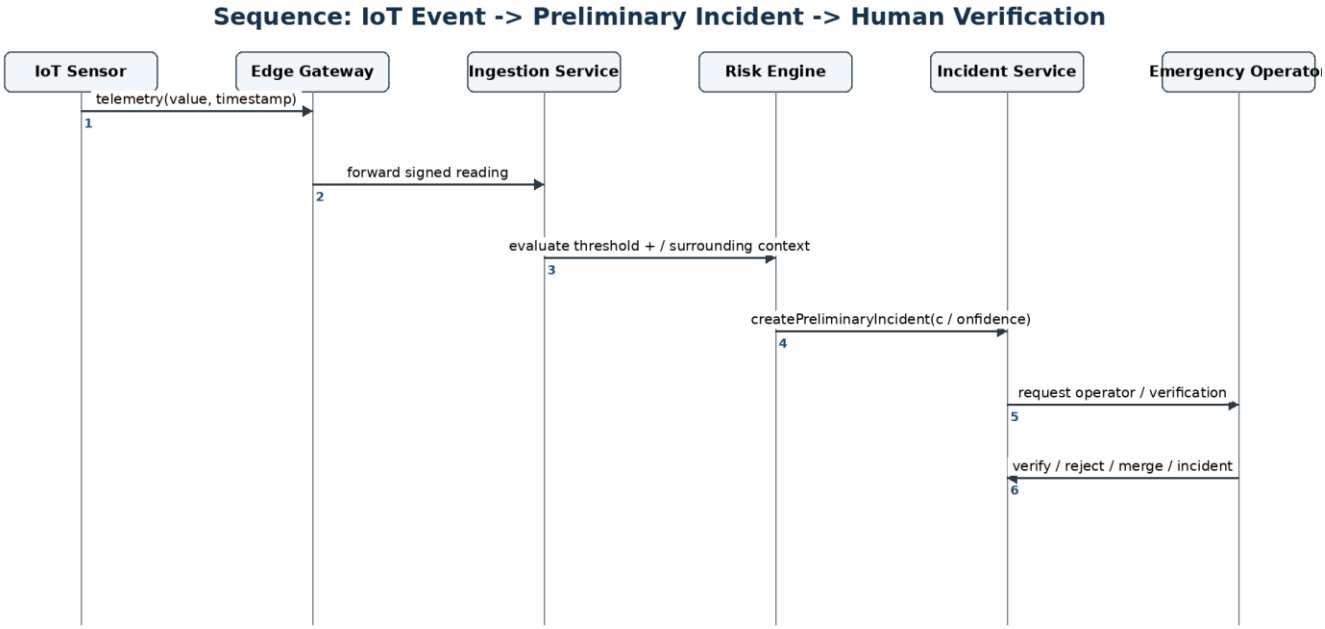

Figure 4. Sequence – Sensor event to preliminary incident verification

IoT data is treated as evidence rather than unquestioned truth. The edge gateway and ingestion service validate and contextualize the reading, the risk engine detects abnormal conditions, and the resulting preliminary incident must still be reviewed by an emergency operator.

9. Activity Diagram

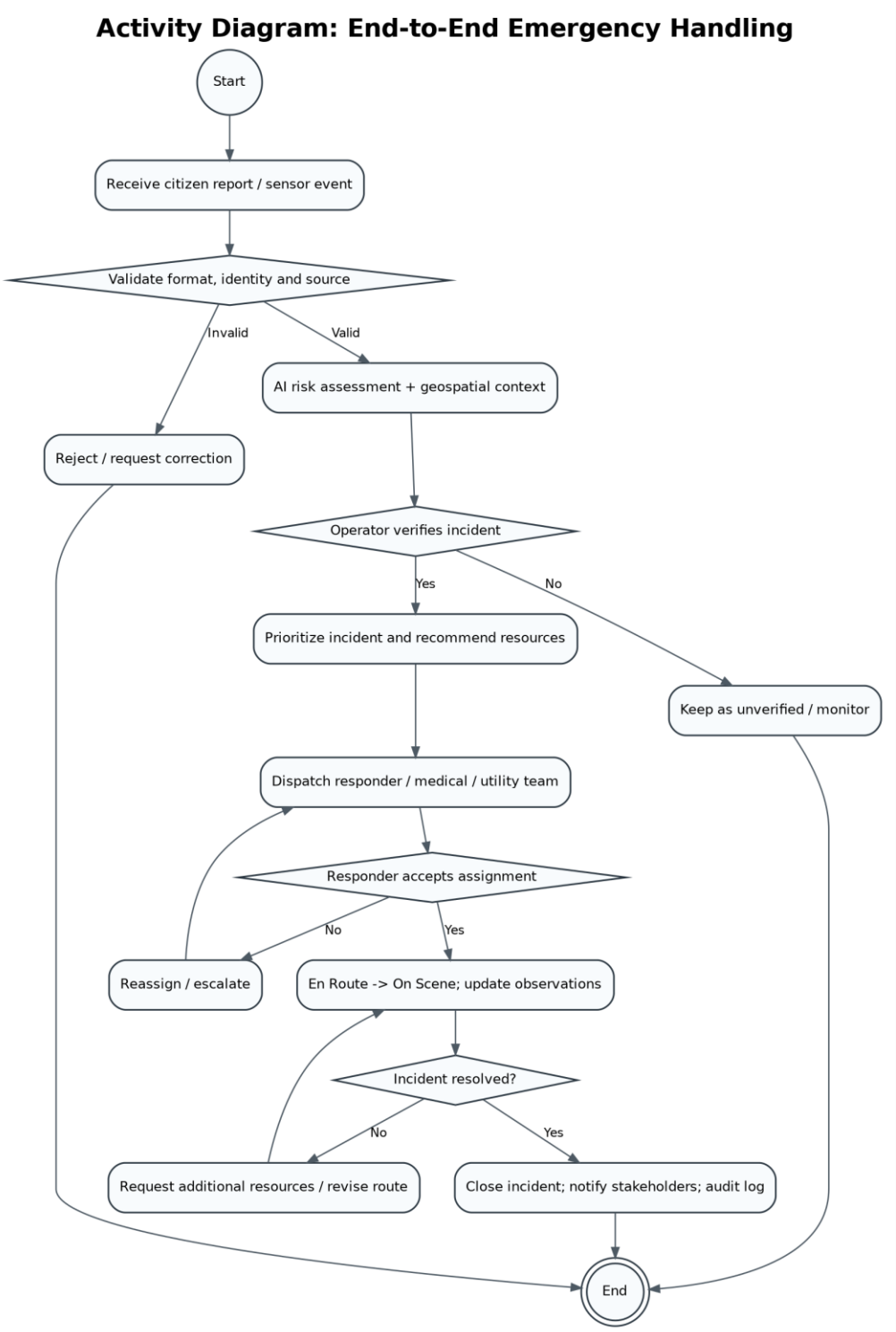

Figure 5. Activity Diagram – End-to-end emergency handling workflow

The activity model shows validation, AI assessment, human verification, prioritization, dispatch, responder acknowledgement, escalation and closure. Rejection and monitoring paths ensure that unverified data cannot silently become an operational alert or dispatch command.

10. State Transition Diagram

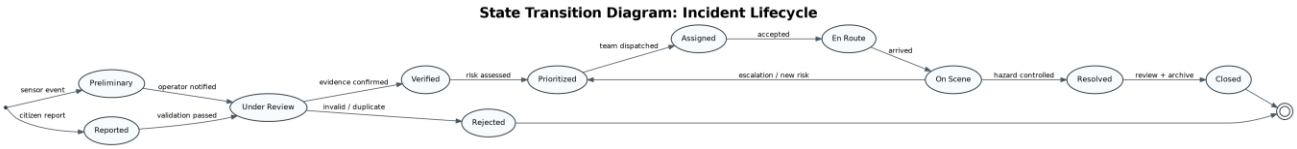

Figure 6. State Transition Diagram – Incident lifecycle

The Incident object progresses through explicit states. Preliminary and Reported incidents enter Under Review; only Verified incidents may be prioritized and assigned. Rejected incidents terminate without operational dispatch, while an On Scene incident may be re-prioritized if new risk information indicates escalation.

11. Package Diagram

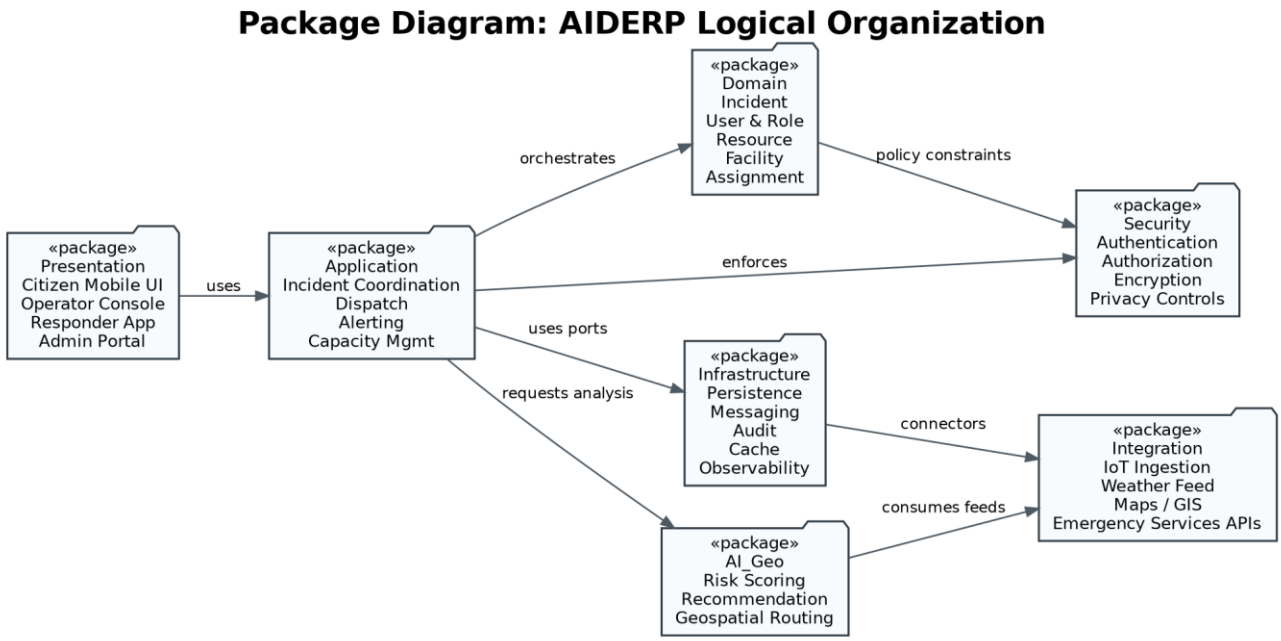

Figure 7. Package Diagram – Logical package organization

The package structure separates user interfaces, application orchestration, domain rules, AI/geospatial decision support, infrastructure concerns, external integration and security. This separation limits change propagation and supports high cohesion within each package.

12. Component Diagram

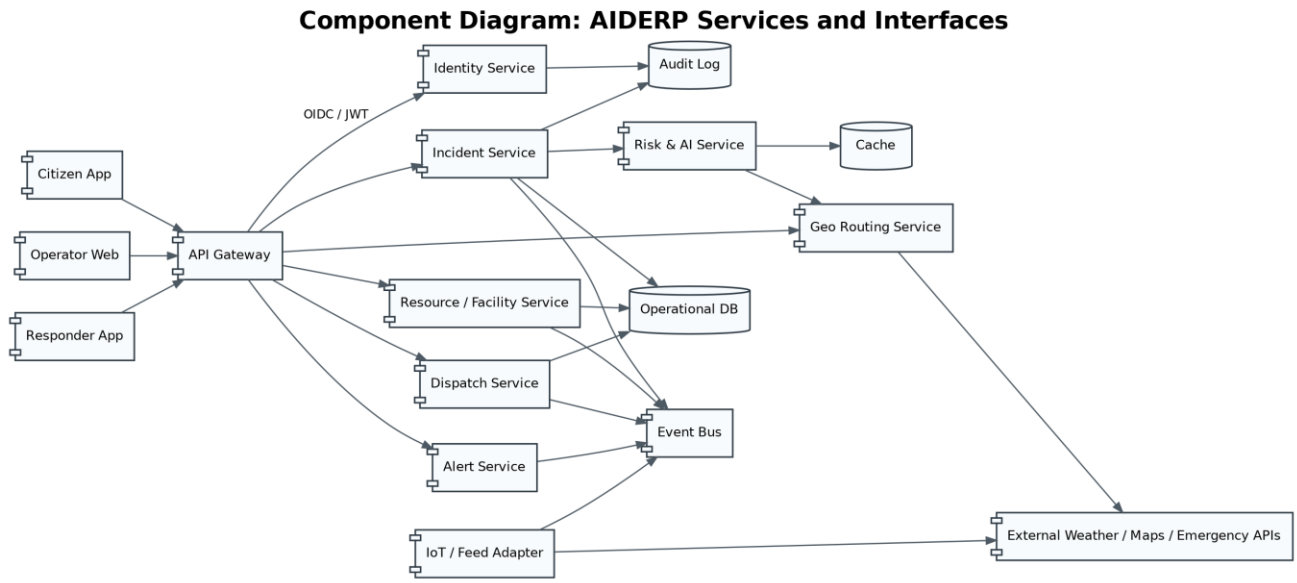

Figure 8. Component Diagram – Services, data stores and external adapters

The component model exposes role-specific clients through an API gateway and isolates identity, incident, dispatch, alert, resource, AI and routing capabilities. An event bus reduces direct coupling between time-critical services and supports asynchronous alerting, sensor ingestion and recovery after temporary failures.

13. Deployment Diagram

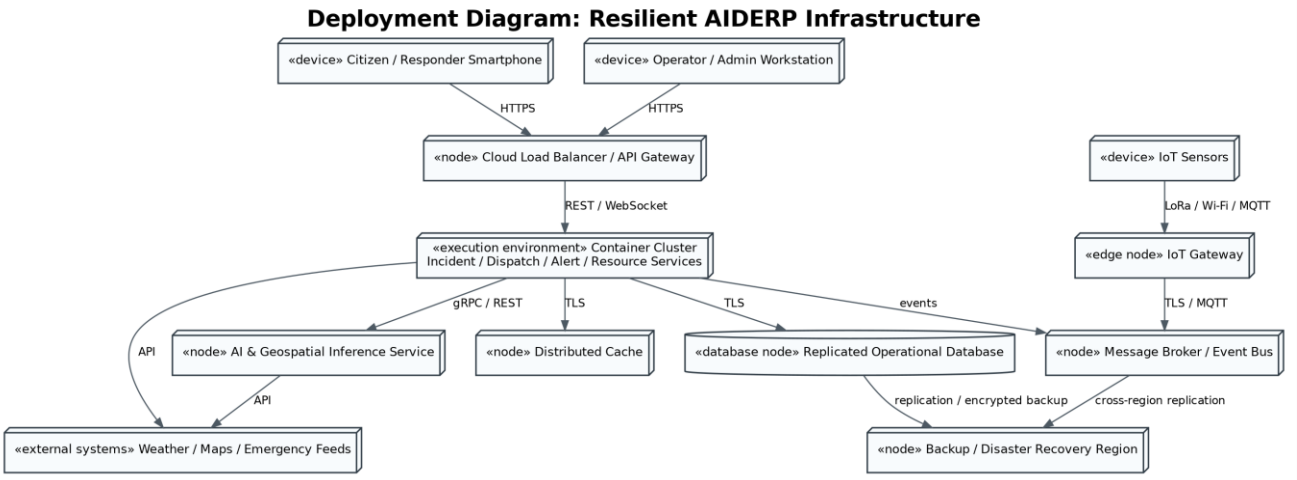

Figure 9. Deployment Diagram – Cloud, edge, device and disaster-recovery nodes

The deployment view includes mobile/desktop clients, IoT edge gateways, a cloud API entry point, horizontally scalable application services, AI/geospatial services, messaging, database/cache infrastructure and an independent disaster-recovery region. Secure transport is required on all external and internal links containing operational or personal data.

14. Object Responsibility Analysis

Class / Object	Primary Responsibilities	Collaborators
Incident	Maintain incident identity, location, type, severity, verification and lifecycle state	Citizen, EmergencyOperator, RiskAssessment, Assignment, Alert
EmergencyOperator	Validate/verify evidence, approve priority, dispatch resources, issue verified alerts	Incident, RiskAssessment, Dispatch/Assignment, Resource
RiskAssessment	Combine incident, exposure, accessibility, resource and real-time feed factors; produce explainable recommendation	Incident, SensorReading, Geo/Weather adapters, Resource
Assignment	Track responder/resource assignment, priority, acceptance and completion	Incident, Responder, Resource
Responder	Accept assignment, update field status, submit observations and	Assignment, Incident, Resource

Class / Object	Primary Responsibilities	Collaborators
	request resources	
Facility	Maintain current capacity/availability and timestamp	Hospital/Shelter user, SafeRoute, Resource service
Alert	Represent a verified message and publication status	Incident, EmergencyOperator, Notification service
SafeRoute	Provide/recalculate route to suitable shelter using hazard and accessibility context	Citizen, Shelter, Geo service

Responsibility assignment follows the Information Expert principle: each object owns behavior closest to the data it controls. Coordination that crosses multiple aggregates is placed in application services rather than forcing domain entities to depend directly on infrastructure or external APIs.

15. Design Axioms / Principles and Design Pattern Justification

Principle	Application in AIDERP
Encapsulation	Incident state changes occur through setStatus()/markVerified() rather than direct field modification, protecting lifecycle invariants.
Abstraction	External weather, map, IoT and emergency systems are accessed through interfaces/adapters so domain logic is not tied to vendor APIs.
Inheritance & LSP	Role classes specialize User only where they can be treated as users; Hospital/Shelter specialize Facility with compatible capacity behavior.
Polymorphism	Notification channels, risk strategies or resource allocation policies can implement common interfaces and be substituted by configuration.
High cohesion	Incident Service manages incident workflow; Alert Service manages alert publication; Geo service manages routing.
Low coupling / DIP	Application services depend on interfaces such as IncidentRepository, GeoProvider and NotificationGateway rather than concrete technologies.
Open/Closed Principle	New hazard types, sensors, notification channels or prioritization strategies can be added through new implementations without

Principle	Application in AIDERP
	rewriting core coordination.
Single Responsibility	Authentication, incident lifecycle, dispatch, AI scoring and external integration are separated into focused components.

Pattern	Where Used	Justification
Strategy	Risk scoring, prioritization and safe-route selection	Different disasters require different scoring/routing policies; strategies can be selected by incident type/context.
Observer / Publish-Subscribe	Alerts, responder updates, capacity changes and sensor events	Multiple consumers react to the same event without tight point-to-point coupling.
Adapter	Weather, maps/GIS, IoT gateways and emergency-service APIs	Normalizes inconsistent external interfaces behind stable internal contracts.
Factory Method	Creation of hazard-specific Incident/RiskStrategy objects	Centralizes creation while supporting new incident types.
Repository / DAO	Incident, user, assignment and resource persistence	Separates domain/application logic from database technology.
Facade	Emergency Coordination API	Provides a simplified application boundary for clients while internal services remain modular.
Circuit Breaker	External feeds and communication gateways	Prevents cascading failure and enables graceful degradation when external systems fail.

16. Assumptions, Constraints, Security and Privacy Considerations

Assumptions

- Authorized operators/responders receive organization-managed identities; citizen identity requirements may vary by local emergency policy.
- External weather/GIS/emergency APIs can be intermittently unavailable, so cached/stale-data indicators are required.
- IoT sensors have identifiable sources and timestamps, but sensor readings can be noisy, compromised or delayed.
- AI models provide decision support with confidence and contributing factors; final verification/dispatch authority remains with authorized humans.

- Location and capacity information is updated frequently enough to support operational decisions, with freshness timestamps visible to users.

Constraints and Resilience Controls

- Verified and unverified information must be visually and logically separated throughout the workflow.
- Core functions must tolerate temporary loss of a feed or region by using retries, queues, cached maps/rules and failover services.
- Conflicting reports should be correlated/merged without deleting original evidence or audit history.
- Assignments must be idempotent so retries do not create duplicate dispatches.
- When AI/geospatial services are unavailable, operators should retain manual priority/dispatch capability with clear degraded-mode indicators.

Security and Privacy

Area	Control
Authentication	MFA for operators/admins; secure citizen session; short-lived tokens; device/session revocation.
Authorization	Role-based and attribute-aware access; least privilege; facility users can update only their own capacity.
Confidentiality	TLS in transit; encryption at rest; secrets stored in a managed vault; sensitive fields masked in logs.
Integrity	Signed sensor messages where available; source/timestamp checks; immutable/tamper-evident audit records.
Privacy	Data minimization for location/health/contact data; explicit purpose, retention and access controls.
Availability	Rate limiting, queueing, autoscaling, multi-zone deployment, backups and tested disaster recovery.
AI governance	Record model/version, inputs, confidence and operator decision; monitor bias/drift; never auto-dispatch solely from opaque output.

17. Requirement-to-Use Case / Class Traceability Matrix

Requirement	Use Case(s)	Supporting Classes	Diagram Trace
FR1 Incident reporting	UC1	Citizen, Incident, SensorReading	Fig. 1, 2, 3, 5
FR2 Assistance request	UC1/UC5	Citizen, Incident, SafeRoute	Fig. 1, 2, 5
FR3 Sensor/feed ingestion	UC7	SensorReading, Incident, RiskAssessment	Fig. 1, 2, 4, 8, 9
FR4 Incident verification	UC2	EmergencyOperator, Incident	Fig. 1, 2, 3, 5, 6
FR5 AI risk assessment	UC2/UC3/UC7	RiskAssessment, Incident, Resource	Fig. 1, 2, 3, 4, 8
FR6 Prioritization/dispatch	UC3	EmergencyOperator, Assignment, Responder, Resource	Fig. 1, 2, 3, 5, 6
FR7 Responder workflow	UC4	Responder, Assignment, Incident	Fig. 1, 2, 3, 5, 6
FR8 Alerts/evacuation	UC5	Alert, SafeRoute, Shelter, Citizen	Fig. 1, 2, 5, 8
FR9 Capacity/resource updates	UC6	Facility, Hospital, Shelter, Resource	Fig. 1, 2, 8
FR10 Identity/audit	All protected use cases	User + security/infrastructure components	Fig. 7, 8, 9

Traceability is maintained by using the same operation names and state concepts across class, sequence, activity and state models. For example, verifyIncident(), dispatchTeam()/assign(), acceptAssignment() and updateStatus() appear consistently in both structural and behavioral views.

18. Implementation / Prototype and Results

Prototype Scope

A practical minimum viable prototype can be implemented as a responsive web/mobile front end connected to REST/WebSocket APIs. The prototype should include four role-based screens: Citizen Incident Report, Emergency Operator Dashboard, Responder

Assignment View and Facility Capacity Update. A lightweight Incident Service stores incident and assignment data; a mock Risk Strategy computes a transparent risk score from severity, population exposure and accessibility; a simulated IoT publisher sends sensor events; and a map adapter returns a safe-route result. This scope is sufficient to demonstrate the UML interactions without pretending that a full emergency-grade production system has already been built.

Suggested Prototype Module Mapping

Prototype Module	Demonstrated Functions	Design Trace
Citizen UI	Report form, location capture, assistance button, alert/shelter view	UC1, UC5
Operator Dashboard	Unverified/verified queues, risk explanation, dispatch control, resource map	UC2, UC3
Responder UI	Assignment details, Accept, En Route, On Scene, Resolved, resource request	UC4
Facility UI	Capacity update and timestamp	UC6
Incident API	Validation, incident state transitions, audit events	Incident class / sequence/activity models
Mock Risk Engine	Rule/strategy-based risk score and recommendation with confidence	RiskAssessment class
Event/Notification Simulator	Sensor readings and alert/responder notifications	Observer/pub-sub pattern

Design-Level Scenario Validation Results

Scenario	Expected / Observed Design Behavior	Result
Citizen reports flood and requests assistance	Report is validated; incident created as unverified; risk assessment triggered; operator queue updated	Pass at design walkthrough level
Operator verifies and dispatches team	Only verified incident reaches prioritization/dispatch; assignment is recorded and responder notified	Pass at design walkthrough level
Responder updates En Route -> On Scene -> Resolved	Transitions match state model and update incident/assignment history	Pass at design walkthrough level
Sensor threshold exceeded	Preliminary incident created; operator verification required before dispatch/verified	Pass at design walkthrough

Scenario	Expected / Observed Design Behavior	Result
	alert	level
External map/weather service unavailable	Circuit breaker/degraded mode uses cached information and manual operator action	Required resilience behavior identified; implementation test pending

The table above reports design/prototype walkthrough outcomes, not production performance measurements. Before deployment, the system would additionally require automated unit/integration tests, security testing, load/failover testing, accessibility testing, incident-response drills and validation with emergency-service stakeholders.

19. Reflection on Design Decisions, SDG Relevance and Learning Outcomes

The most important design decision was to separate information collection from operational authority. AIDERP may accept citizen reports, sensor events and AI recommendations rapidly, but these inputs do not become verified operational facts automatically. This distinction improves safety, accountability and traceability. Modeling the system through use cases, classes, interactions, activities and states also demonstrated why consistency across UML views matters: a method such as verifyIncident() is meaningful only when its preconditions, state transition and downstream effects agree across diagrams.

The package, component and deployment views reinforced the value of modularity and interface-based design. Disaster platforms must integrate changing external feeds while surviving partial failures, so adapters, events, circuit breakers and replicated infrastructure are more appropriate than a tightly coupled monolithic workflow. The OOAD principles of encapsulation, high cohesion, low coupling, substitution and open/closed extension directly support maintainability and resilience.

SDG Relevance

SDG	Connection to AIDERP
SDG 3 – Good Health	Rapid assistance requests, medical coordination and hospital-

SDG	Connection to AIDERP
and Well-being	capacity visibility can improve emergency response and continuity of care.
SDG 9 – Industry, Innovation and Infrastructure	IoT, resilient cloud/edge architecture and interoperable services strengthen emergency infrastructure and innovation.
SDG 11 – Sustainable Cities and Communities	Verified alerts, shelter guidance, safe routing and coordinated response support safer and more resilient communities.
SDG 13 – Climate Action	Flood, cyclone, heatwave and fire monitoring/response capabilities support adaptation and climate-risk resilience.

Learning outcome: this assignment applies object-oriented analysis from problem understanding through class/responsibility identification, then connects those concepts to UML behavioral and architectural models. It also demonstrates that design quality is not only diagram correctness; it includes security, privacy, failure behavior, traceability and the ability to evolve the system safely.

20. GitHub Upload

The final submission can be uploaded to GitHub as a structured repository. The repository should contain the report, UML source files (for example PlantUML/StarUML/Draw.io), exported diagrams, and any prototype code. A README should summarize the problem, actors, architecture, setup steps and traceability to the assignment deliverables.

```
AIDERP-OOAD/
├── README.md
├── docs/
│   ├── AIDERP_Assignment_Report.docx
│   └── traceability-matrix.md
├── uml/
│   ├── use-case/
│   ├── class/
│   ├── sequence/
│   ├── activity-state/
│   └── architecture/
├── prototype/
│   ├── frontend/
│   └── backend/
└── tests/
```

GitHub Repository URL: _____

Before submission, verify that no secrets, personal credentials, API keys or sensitive emergency data are committed. Use a .gitignore file for environment files and generated build artifacts.

— End of Answer —